Project Documentation

Project Title:LibSpace-A Robust Backend for Library

Operations

Team Leader : RAHMATH NISHA N

(nisharahmath546@gmail.com)

Team members :

[SHAFRAN B , shafrans302@gmail.com]

[NIVETHA DHARSHINI S , nivethadharshini142@gmail.com]

[ARUL LIYONSHIYA S, liyonshiya@gmail.com]

1. Introduction:

• Project Title:LibSpace-A Robust Backend for

Library Operations

2. Project Overview:

Purpose:

The purpose of Libspace is to develop a backend system

that manages library book records and user accounts

securely.It demonstrates user authentication,secure API

development,database integration,and CRUD operations.

Features:

• User registration and login

• Secure password hashing using bcrypt

• JWT-based authentication

• Add,update,and delete books

• View book list and get book by ID • One user can

manage multiple books

3. Architecture:

API Layer(Routes)

Handles incoming HTTP requests and API endpoints using

Node.js and Express.js.

• /api/auth/register

• /api/auth/login

• /api/book/add

• /api/book/:id

Business Logic Layer(Controllers)

• Controllers implement core application logic including

authentication,password hashing,JWT generation,and

CRUD operations.

Data Access Layer(Database/Models)

• The data layer manages database interaction using

Mongodb and Mongoose.

• Models represent Users and Books collections.

4. Technologies Used:

• Node.js

• Express.js

• MongoDB

• Mongoose

• JWT(JSON Web Token)

• bcrypt

• Thunder client • Visual Studio Code 5. Installation:

• Install Dependencies npm install

• Environment Variables(.env)

PORT=5000

MONGO_URI=mongodb;//127.0.0.1;27017/libs

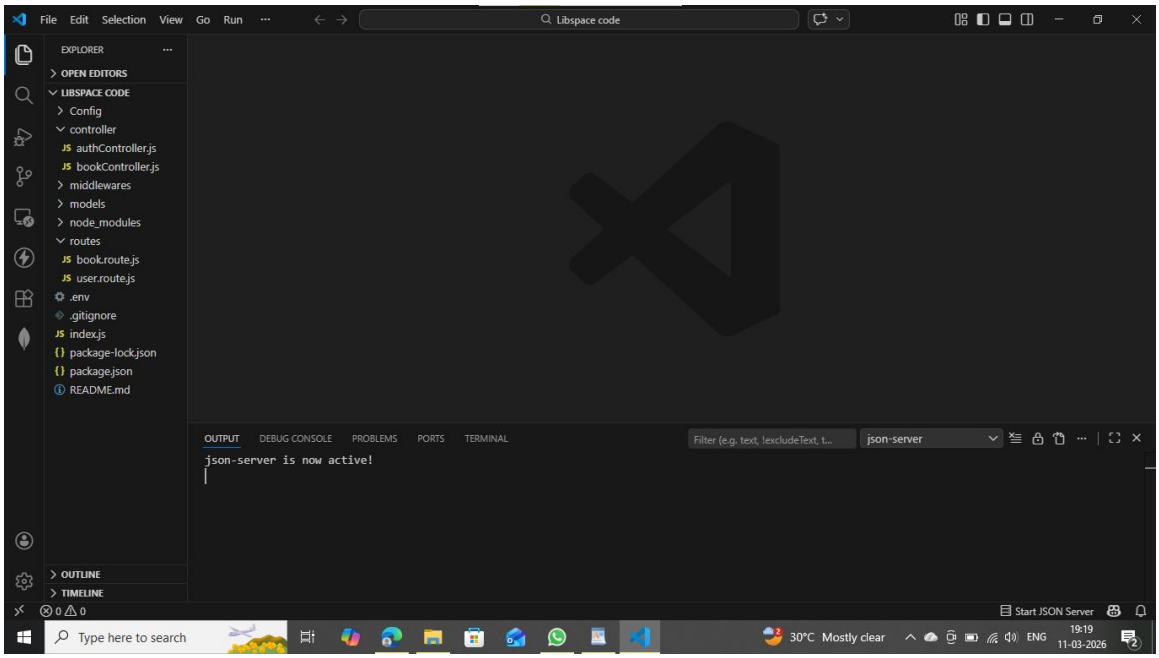
pace

JWT_SECRET=your_secret_key

• Run the Server npm start or

npm run dev

6. Folder Structure:



7. API Documentation:

Users APIs

• POST/api/auth/register

• POST/api/auth/login

• DELETE/api/auth/delete

• POST/api/auth/logout

Book APIs

• POST/api/book/add

• GET/api/book

• GET/api/book/:id

• PUT/api/book/:id

• DELETE/api/book/:id

8. Authentication:

LibSpace uses JWT for authentication.After login,the

server generates a token which must be sent in request

headers for protected routes.

Authorization: Bearer <token>

9. Database Design

Users Collection Fields

• name

• email

• password

• createdAt

• updatedAt

Books Collection Fields

• title

• author

• url

• user(Objected reference)

10.Testing:

1. Tested using Thunder client

2. Verified using registration and login

3. Verified book CRUD operations

4. Verified token authentication

11.Screenshots:

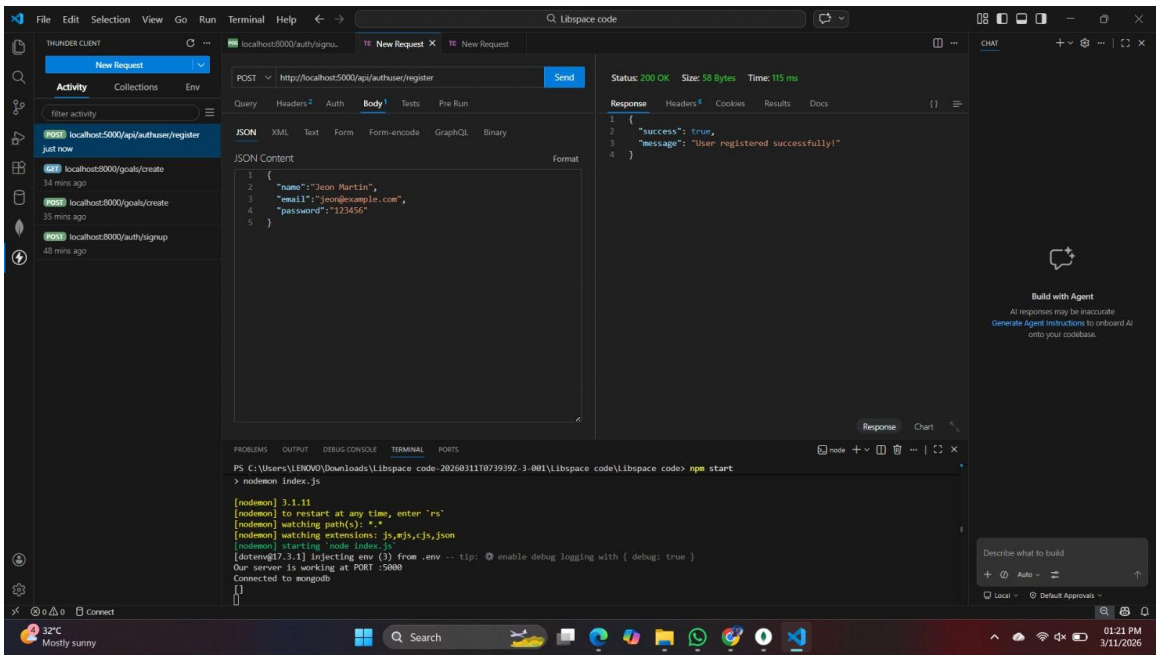


Fig 1.1 Successful Connection to MongoDB

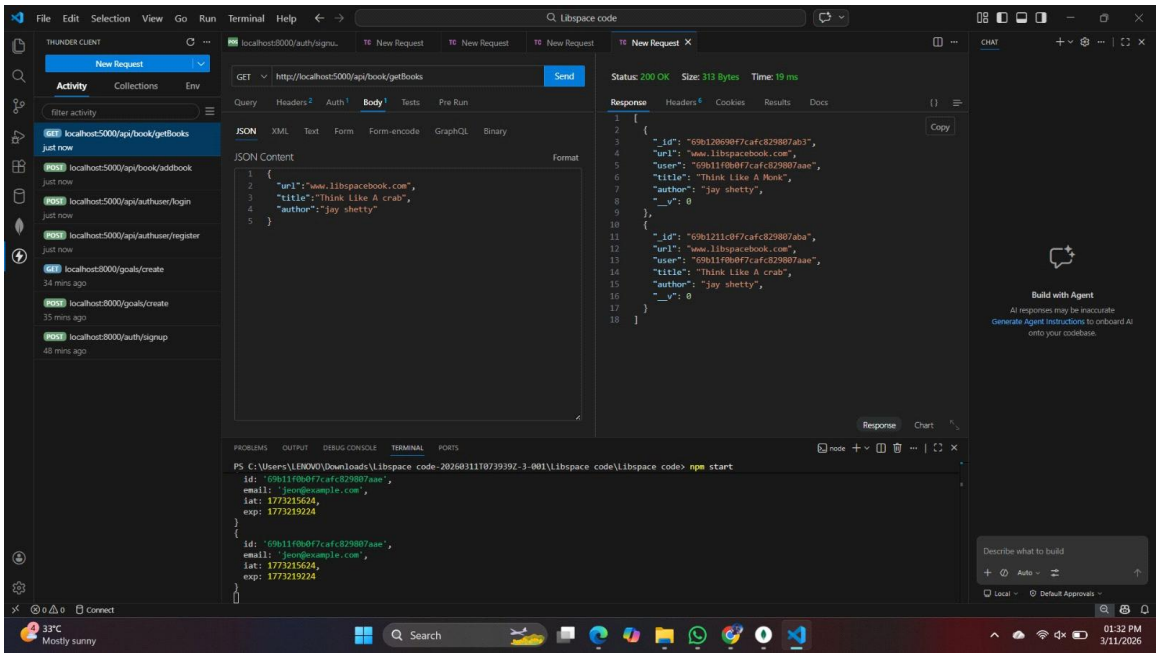


Fig 1.2 Displaying Book

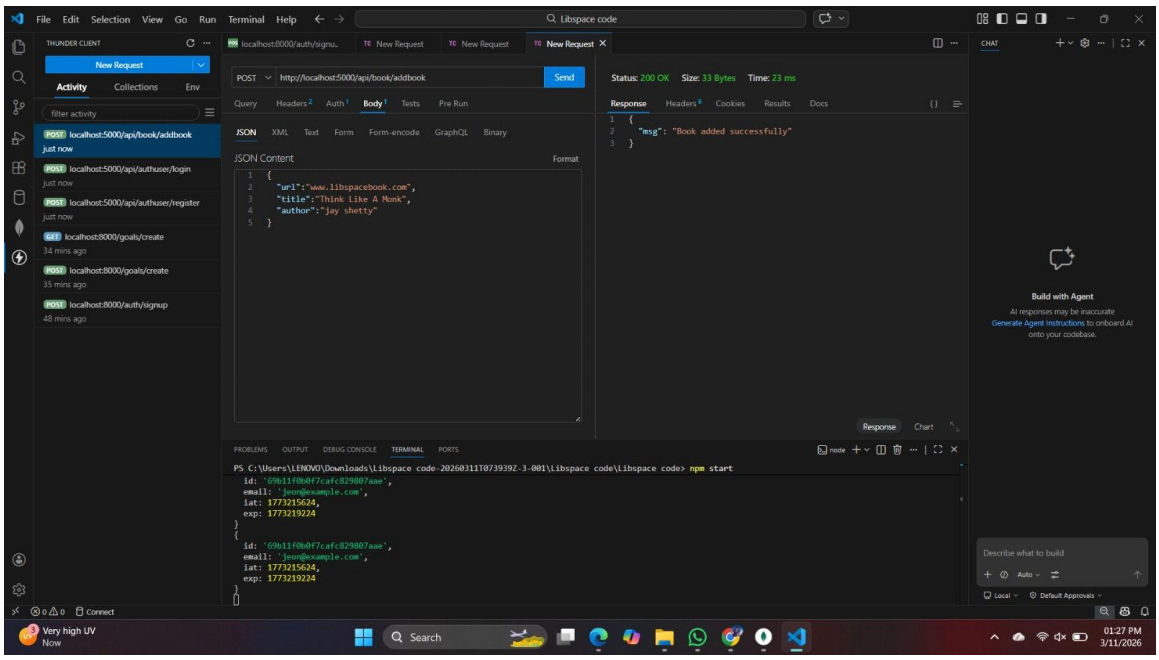


fig:1.3 adding new book

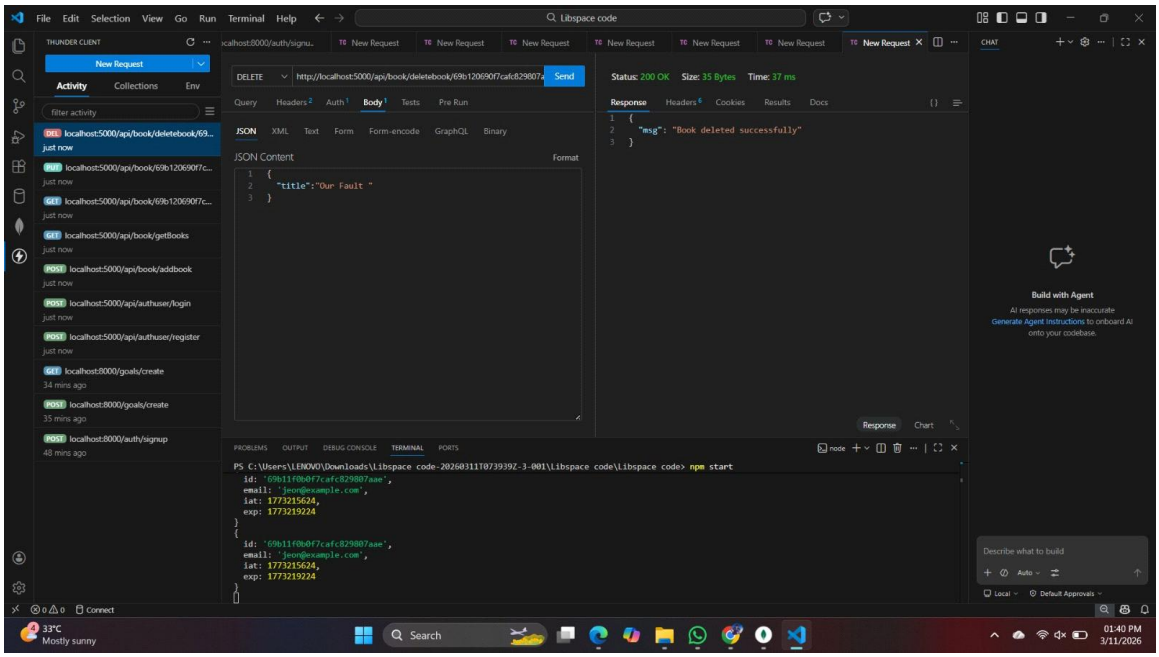


fig:1.4 Deleting book

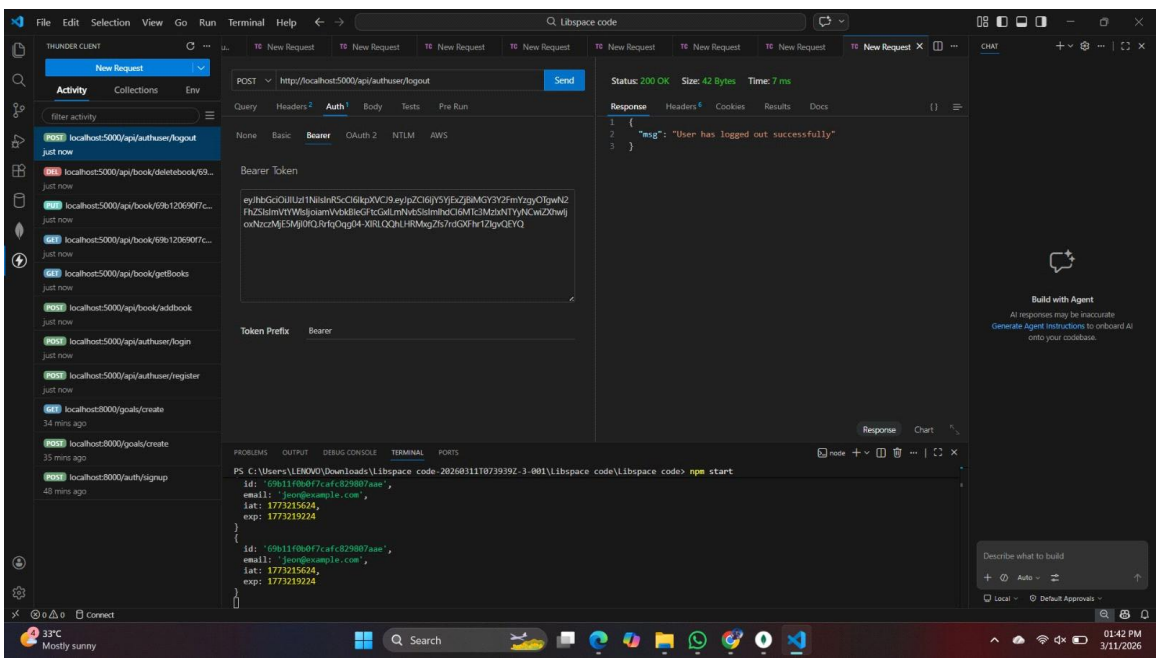


Fig 1.5 User Logout Response

12.Known Issues:

• No frontend interface

• Requires Thunder client for testing • Token blacklist

optimization needed

13.Future Enhancements:

• Frontend integration with React

• Role-based authentication

• Cloud database deployment

• Book file upload support • Mobile application

support

14.Demo Video Link:
https://drive.google.com/file/d/1tQOhbxEiDArjeL8oep_NMc84JU31zPSQ/view?usp=drivesdk

15. Libspace Project
code:https://drive.google.com/drive/folders/1RB5C_Ba1YAP-Z_B15nIvYMoelRkbI0yg